

Assignment 3: SQL Injection

Nouf Alquhiadan 445008277

Objective

- Understand how SQL Injection works
- Identify vulnerable code
- Observe how input changes database queries
- Learn how to fix vulnerability

Lab Setup

Step 1: Create Lab Folder

```
mkdir -p /sqli_lab  
cd /sqli_lab
```

screen shot :

A terminal window screenshot showing the following commands and output:

```
(nouf@kali2)-[~]  
$ sudo su  
[sudo] password for nouf:  
(root@kali2)-[/home/nouf]  
# mkdir -p /sqli_lab  
cd /sqli_lab  
  
(root@kali2)-[/sqli_lab]  
#
```

Step 2: Create Vulnerable Python App

```
nano app.py
```

Paste this code:

```
from flask import Flask, request  
import sqlite3  
app = Flask(__name__)  
# Create database  
def init_db():  
    conn = sqlite3.connect('users.db')  
    c = conn.cursor()  
    c.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY  
KEY, username TEXT, password TEXT)")
```

```

        c.execute("INSERT INTO users (username, password) VALUES ('admin',
'1234')")
        c.execute("INSERT INTO users (username, password) VALUES ('user',
'pass')")
        conn.commit()
        conn.close()
init_db()
@app.route("/")
def login_form():
    return '''
    <h2>Login Page</h2>
    <form action="/login" method="post">
        Username: <input name="username"><br>
        Password: <input name="password"><br>
        <button type="submit">Login</button>
    </form>
    '''
@app.route("/login", methods=["POST"])
def login():
    username = request.form['username']
    password = request.form['password']
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    # Vulnerable query
    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
    result = c.execute(query).fetchone()
    conn.close()
    if result:
        return "<h3>Login Successful</h3>"
    else:
        return "<h3>Login Failed</h3>"
if __name__ == "__main__":
    app.run(debug=True)

```

Screen shot :

```
root@kali2: /sql_lab
File Actions Edit View Help
GNU nano 6.2 app.py
from flask import Flask, request
import sqlite3

app = Flask(__name__)

def init_db():
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)")
    c.execute("INSERT INTO users (username, password) VALUES ('admin', '1234')")
    c.execute("INSERT INTO users (username, password) VALUES ('user', 'pass')")
    conn.commit()
    conn.close()

init_db()

@app.route("/")
def login_form():
    return """
    <h2>Login Page</h2>
    <form action="/login" method="post">
        Username: <input name="username"><br>
        Password: <input name="password"><br>
        <button type="submit">Login</button>
    </form>
    """

@app.route("/login", methods=["POST"])
def login():
    username = request.form['username']
    password = request.form['password']
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    result = c.execute(query).fetchone()
    conn.close()
    if result:
        return "<h3>Login Successful</h3>"
    else:
        return "<h3>Login Failed</h3>"

if __name__ == "__main__":
    app.run(debug=True)
```

Step 3: Run the App

```
python3 app.py
```

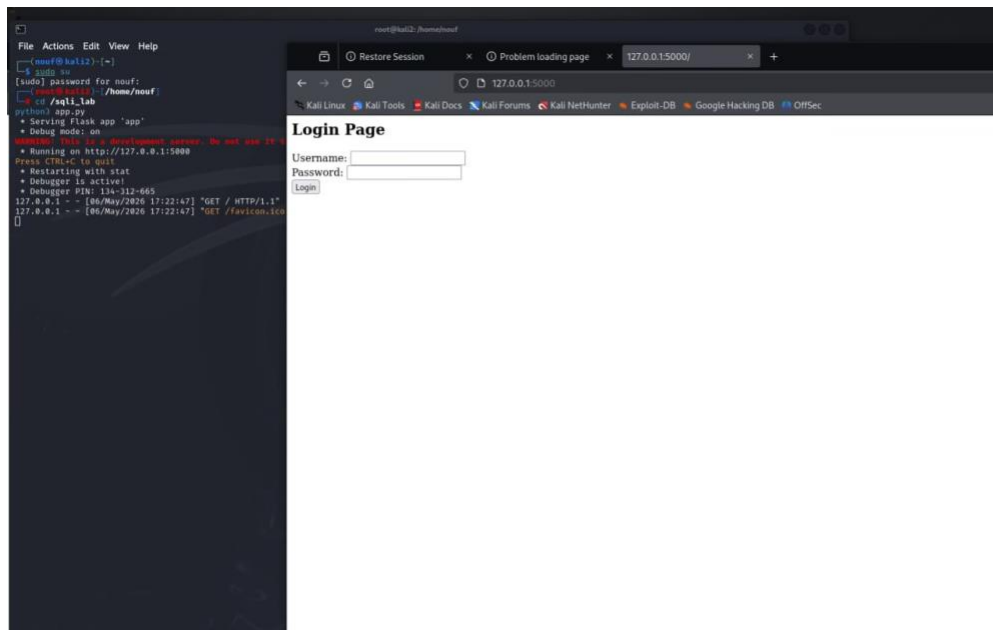
Screen shot :

```
(root@kali2) ~/sql_lab
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 134-312-665
```

Step 4: Open in Browser

```
http://127.0.0.1:5000
```

screen shot :



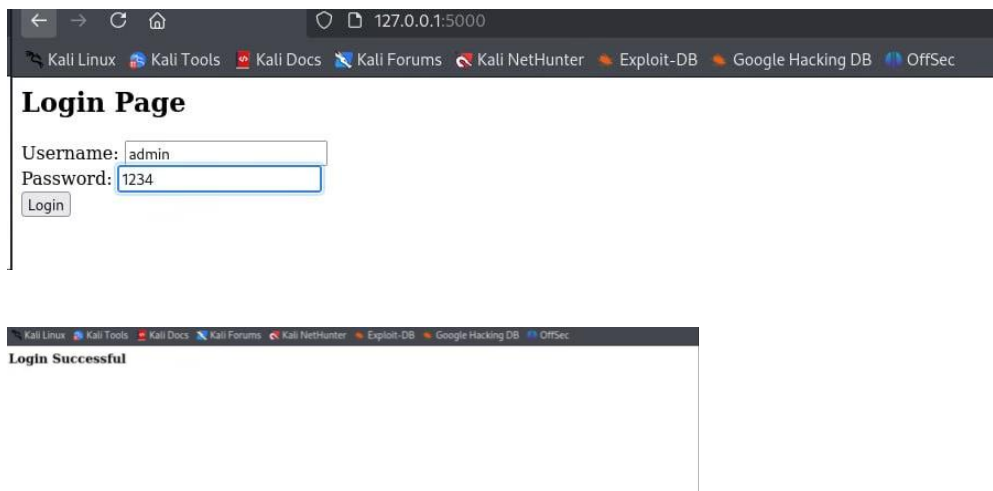
Step 5: Normal Login Test

Try:

- Username: admin
- Password: 1234

✓ Login works normally

Screen shot :

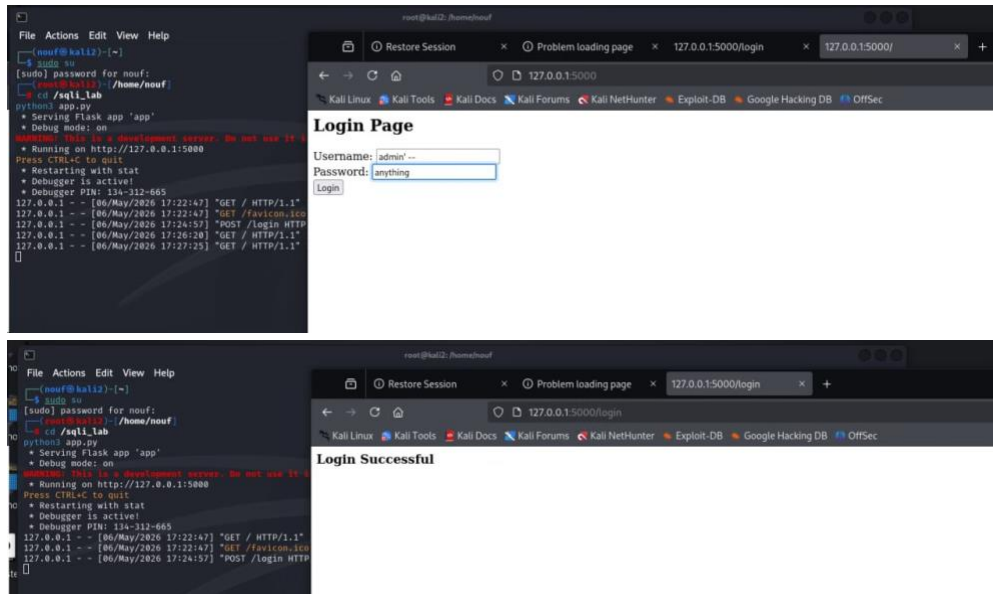


Step 6: Perform SQL Injection (Educational Demo)

Input:

- Username: admin' --
- Password: anything

Screen shot :



What Happens?

The SQL query becomes:

```
SELECT * FROM users WHERE username='admin' -- ' AND password='anything'
```

Everything after -- is treated as a comment and ignored

The password condition is bypassed

Result:

Login Successful **without a password**

This is an example of **SQL Injection**

Explanation for Students

- The application inserts user input directly into the SQL query
- There is no validation or protection
- The attacker manipulates the logic of the query

Step 7: Fix the Vulnerability

Open the file:

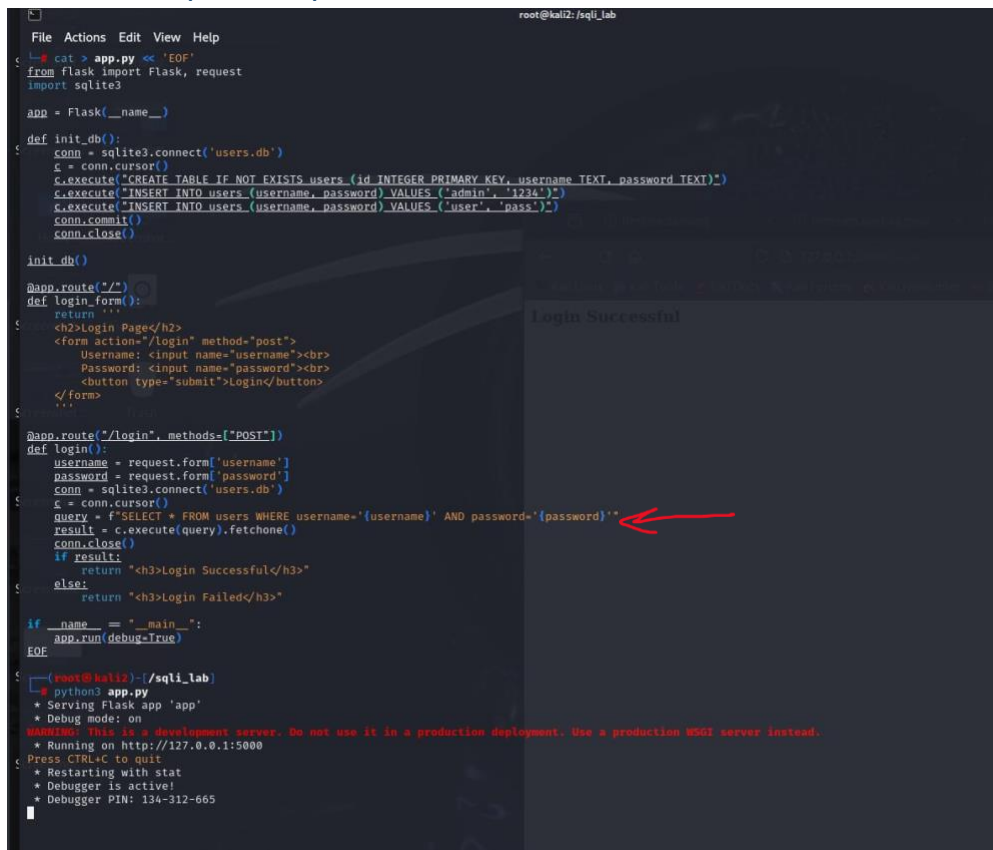
```
nano app.py
```

Replace the vulnerable code with:

```
query = "SELECT * FROM users WHERE username=? AND password=?"
result = c.execute(query, (username, password)).fetchone()
```

Using parameterized queries (prepared statements) ensures that user input is treated as data, not as SQL code. The ? placeholders are safely handled by the database driver, making SQL injection impossible.

Screen shot (Fix code):



```
File Actions Edit View Help
root@kali2: /sql_lab

cat > app.py << EOF
from flask import Flask, request
import sqlite3

app = Flask(__name__)

def init_db():
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)")
    c.execute("INSERT INTO users (username, password) VALUES ('admin', '1234')")
    c.execute("INSERT INTO users (username, password) VALUES ('user', 'pass')")
    conn.commit()
    conn.close()

init_db()

@app.route("/")
def login_form():
    return """
    <h2>Login Page</h2>
    <form action="/login" method="post">
      Username: <input name="username"><br>
      Password: <input name="password"><br>
      <button type="submit">Login</button>
    </form>
    """

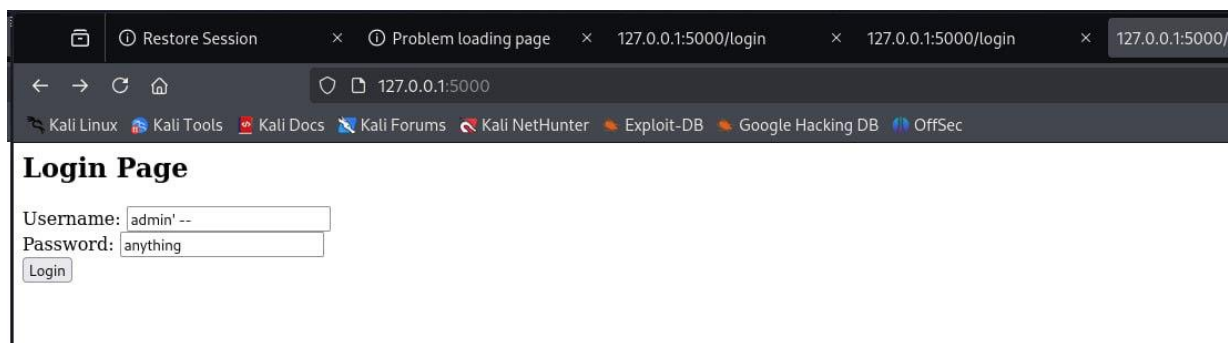
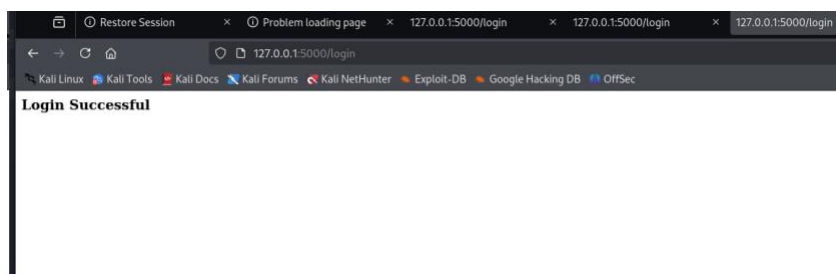
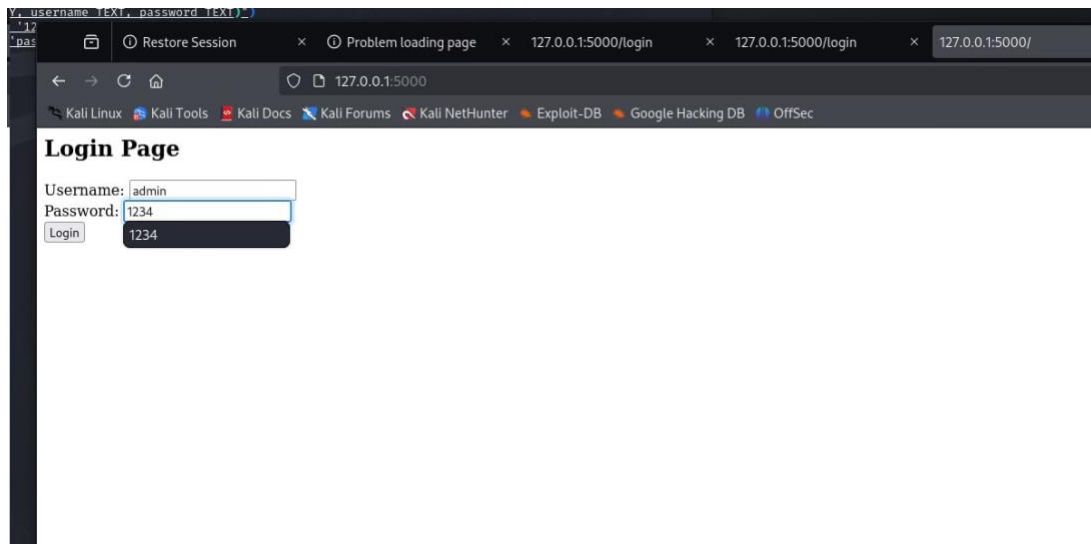
@app.route("/login", methods=["POST"])
def login():
    username = request.form['username']
    password = request.form['password']
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    result = c.execute(query).fetchone()
    conn.close()
    if result:
        return "<h3>Login Successful</h3>"
    else:
        return "<h3>Login Failed</h3>"

if __name__ == "__main__":
    app.run(debug=True)
EOF

python3 app.py
* Serving Flask app 'app'
* Debug modes on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 134-312-665

Login Successful
```

Screen shot (Login Failed after injection attempt — fix works):



Extended Practical Lab: SQL Injection (Advanced)

Objective

- Understand advanced SQL Injection techniques
- Perform Boolean-based SQLi
- Perform Time-based SQLi
- Use automated tools (SQLMAP)
- Learn how attackers extract information

Part 1: Boolean-Based SQL Injection

Concept

This technique uses TRUE / FALSE conditions to analyze the application behavior.

Experiment

✓ TRUE condition:

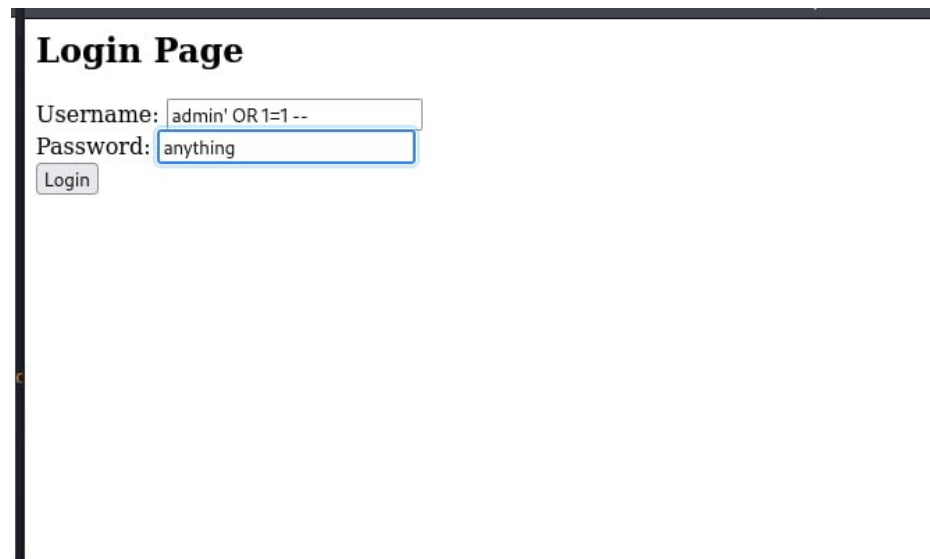
Enter in username:

```
admin' OR 1=1 --
```

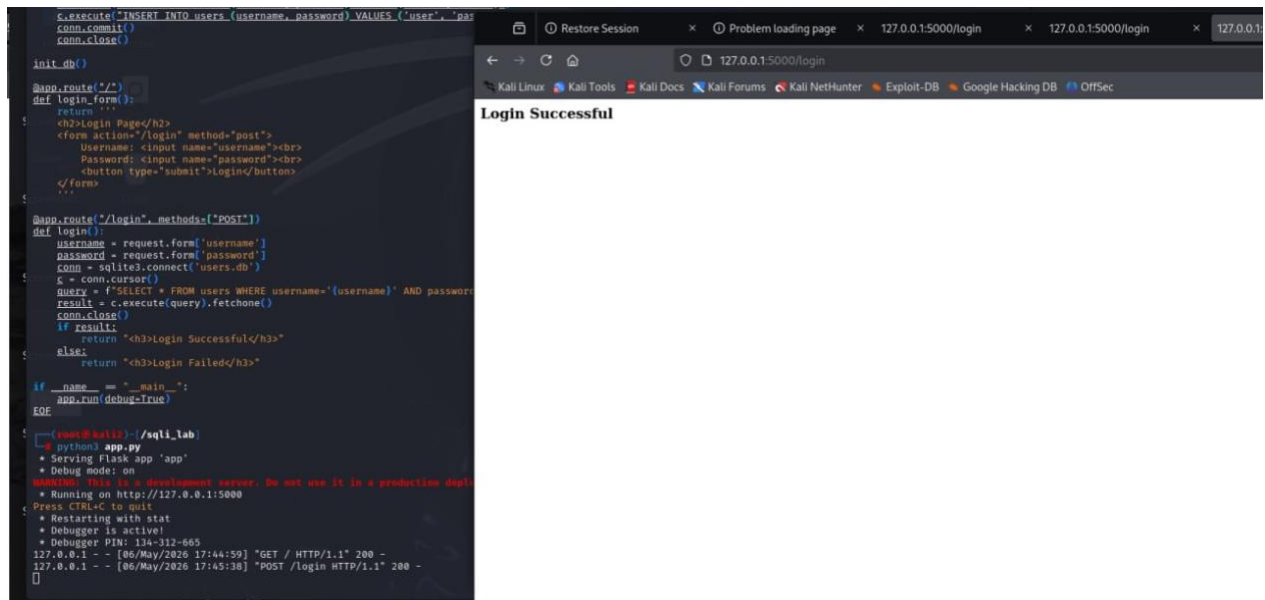
Result: ✓ Login Successful

Since $1=1$ is always TRUE, the WHERE condition evaluates to true and returns the first user record, granting access without a real password.

Screen shot :



The screenshot shows a web application's login page. The title is "Login Page". There are two input fields: "Username:" and "Password:". The "Username:" field contains the text "admin' OR 1=1 --". The "Password:" field contains the text "anything". Below the input fields is a "Login" button. The page is displayed within a browser window with a dark border.



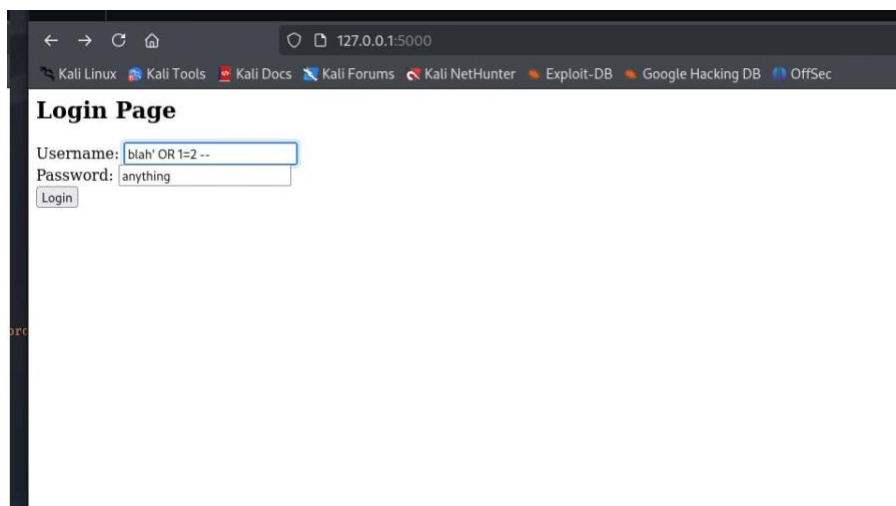
FALSE condition:

blah' OR 1=2 --

Result: Login Failed

Since $1=2$ is always FALSE, the WHERE condition evaluates to false and no record is returned, so access is denied.

Screen shot :





Part 2: Time-Based SQL Injection

Concept

Instead of visible results, the attacker uses response delay.

Modify the Code

Open:

```
nano app.py
```

Add:

```
from flask import Flask, request
import sqlite3
import time    # added for delay simulation
app = Flask(__name__)
# Create database
def init_db():
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)")
    c.execute("INSERT INTO users (username, password) VALUES ('admin', '1234')")
    c.execute("INSERT INTO users (username, password) VALUES ('user', 'pass')")
    conn.commit()
    conn.close()
init_db()
# Home page (login form)
@app.route("/")
def login_form():
    return '''
<h2>Login Page</h2>
<form action="/login" method="post">
    Username: <input name="username"><br>
    Password: <input name="password"><br>
    <button type="submit">Login</button>
</form>'''
```

```

'''
# Login function (VULNERABLE)
@app.route("/login", methods=["POST"])
def login():
    username = request.form['username']
    password = request.form['password']
    # Time-based simulation
    if "delay" in username:
        time.sleep(5)
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    # Vulnerable query
    query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
    result = c.execute(query).fetchone()
    conn.close()
    if result:
        return "<h3>Login Successful</h3>"
    else:
        return "<h3>Login Failed</h3>"
# Run app
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)

```

How to Run

```
python3 app.py
```

Open:

```
http://127.0.0.1:5000
```

Screen shot :

```
(root@kali2)-[/sql_lab]
# cat > app.py << EOF
from flask import Flask, request
import sqlite3
import time

app = Flask(__name__)

def init_db():
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    c.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, username TEXT, password TEXT)")
    c.execute("INSERT INTO users (username, password) VALUES ('admin', '1234')")
    c.execute("INSERT INTO users (username, password) VALUES ('user', 'pass')")
    conn.commit()
    conn.close()

init_db()

@app.route("/")
def login_form():
    return '''
    <h2>Login Page</h2>
    <form action="/login" method="post">
        Username: <input name="username"><br>
        Password: <input name="password"><br>
        <button type="submit">Login</button>
    </form>
    '''

@app.route("/login", methods=["POST"])
def login():
    username = request.form['username']
    password = request.form['password']
    if "delay" in username:
        time.sleep(5)
    conn = sqlite3.connect('users.db')
    c = conn.cursor()
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    result = c.execute(query).fetchone()
    conn.close()
    if result:
        return "<h3>Login Successful</h3>"
    else:
        return "<h3>Login Failed</h3>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
EOF
(root@kali2)-[/sql_lab]
```

Experiment (Time-Based SQLi Simulation)

Input:

delay

Result:

The page takes **5 seconds to respond**. The server delayed because "delay" was detected in the username field, triggering `time.sleep(5)`. This simulates how an attacker uses response timing to confirm injection without any visible output.

Screen shot :

